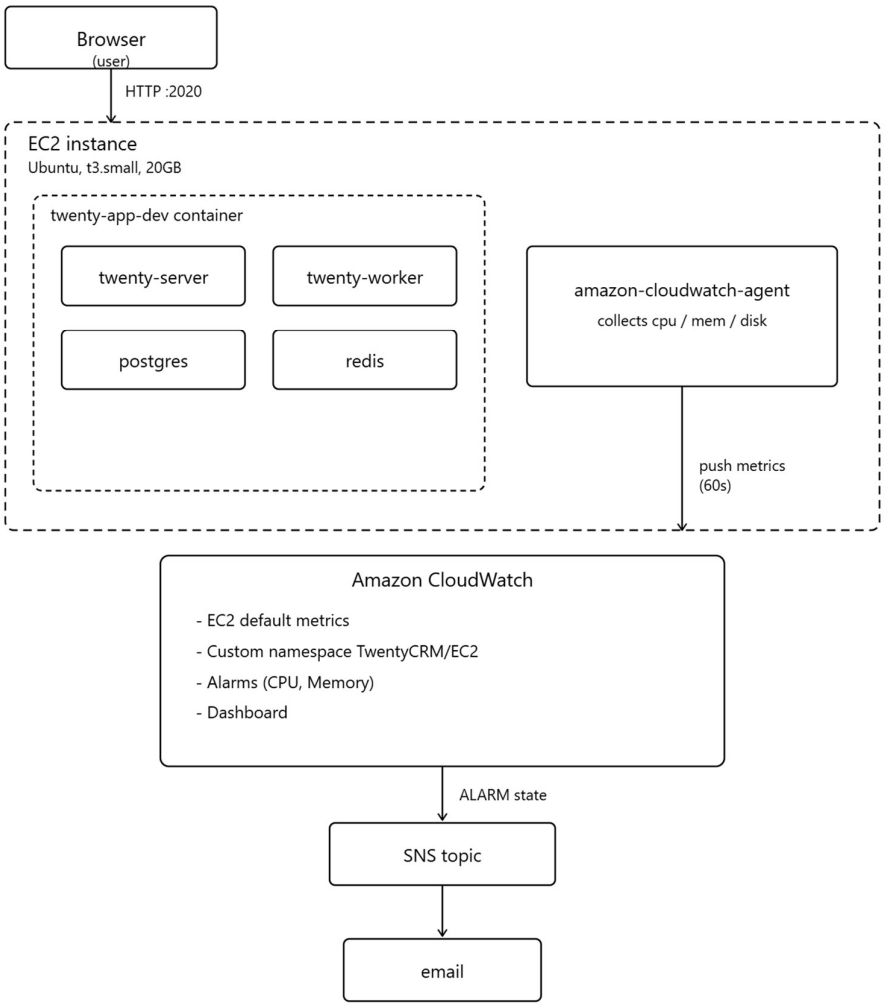
AWS Observability with CloudWatch

---

1. Objective

Deploy the Twenty CRM application on an EC2 instance and implement full observability for it using Amazon CloudWatch — default EC2 metrics, a custom CloudWatch Agent for OS-level metrics (CPU, memory, disk), alarms, and a monitoring dashboard. Generate real traffic against the app and confirm the whole pipeline (metrics → dashboard → alarms → notification) reacts correctly.



---

3. Step-by-Step Implementation

3.1 Launch the EC2 instance

1. AWS Console → EC2 → Launch Instance.
2. **Name:** twenty-crm-<yourname>
3. **AMI:** Ubuntu Server 22.04/24.04 LTS
4. **Instance type:** t3.small
5. **Key pair:** created new key pair, downloaded .pem
6. **Storage:** changed from default 8 GiB to **20 GiB gp3**
7. **Security group inbound rules:**
    - SSH (22) — My IP
    - Custom TCP 2020 — 0.0.0.0/0 (Twenty CRM app port)
8. **IAM instance profile:** CloudWatchAgentEC2Role
    - Trusted entity: AWS service → EC2
    - Attached policy: CloudWatchAgentServerPolicy (lets the CloudWatch Agent push metrics)
    - Attached at launch time so no later stop/attach was needed.
9. Launched the instance.

3.2 Connect to the instance

chmod 400 twenty-crm-key.pem

ssh -i twenty-crm-key.pem ubuntu@<EC2_PUBLIC_IP>

3.3 Deploy Twenty CRM

Initial attempt used raw docker compose with a plain docker-compose.yml from an earlier branch — this repeatedly failed (see **Issues Faced** section). The working path was:

git clone https://github.com/PearlThoughts-Intern-DevOps/devops-crm-project.git

cd devops-crm-project

git checkout fiza

- Pulled the twentycrm/twenty-app-dev Docker image

- Started a single container (twenty-app-dev) bundling Postgres, Redis, the Twenty server, and worker processes (managed internally by s6-overlay)
- Exposed the app on **port 2020**

3.4 Verify Twenty CRM is running

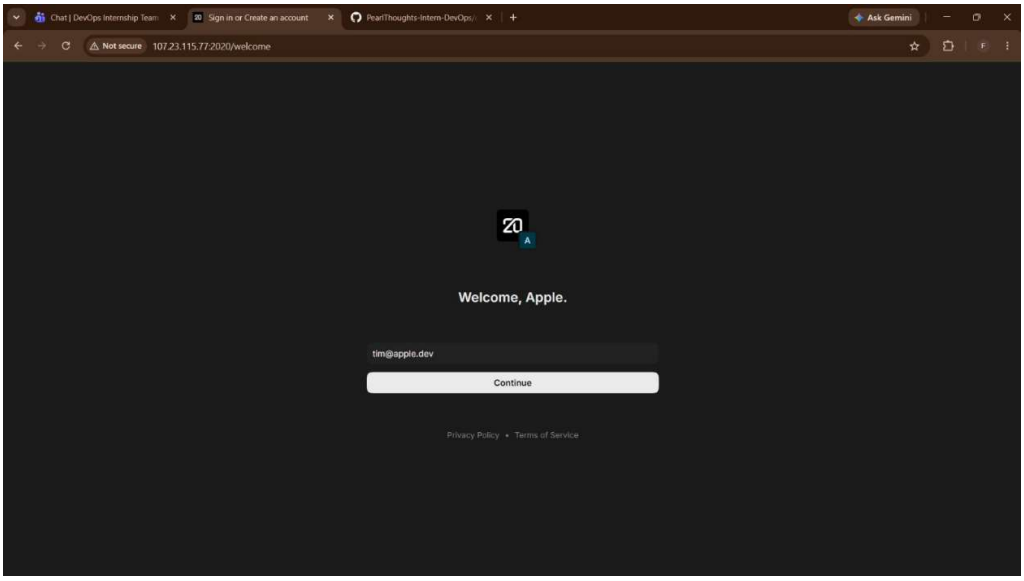
docker ps

curl -s -o /dev/null -w "%{http_code}\n" http://localhost:2020/healthz

Once the healthcheck returned 200 consistently, opened in browser:

http://<EC2_PUBLIC_IP>:2020

Confirmed the Twenty CRM login/sign-up screen loaded. Signed up, created a workspace, and added sample contacts — this also served as the "generate activity" step later.



3.5 Explore default EC2 metrics in CloudWatch

AWS Console → CloudWatch → Metrics → All metrics → EC2 → Per-Instance Metrics → selected the instance.

3.6 Install and configure the CloudWatch Agent

Installed the Debian package directly (chose manual install over the console's "Fleet Manager" flow because the instance's SSM Agent was not registering — see Issues Faced):

wget https://s3.amazonaws.com/amazoncloudwatch-agent/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb

sudo dpkg -i -E ./amazon-cloudwatch-agent.deb

Wrote the agent configuration directly (bypassing the interactive wizard for reliability) targeting a custom namespace so the metrics are easy to find and don't mix with the default EC2 namespace:

```
sudo tee /opt/aws/amazon-cloudwatch-agent/bin/config.json > /dev/null << 'EOF'
{
  "agent": {
    "metrics_collection_interval": 60
  },
  "metrics": {
    "namespace": "TwentyCRM/EC2",
    "metrics_collected": {
      "cpu": {
        "measurement": ["cpu_usage_idle", "cpu_usage_user", "cpu_usage_system"],
        "totalcpu": true
      },
      "mem": {
        "measurement": ["mem_used_percent"]
      },
      "disk": {
        "measurement": ["used_percent"],
        "resources": ["/"]
      }
    }
  }
}
EOF
```

Applied the config and started the agent:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
  -a fetch-config -m ec2 \
  -c file:/opt/aws/amazon-cloudwatch-agent/bin/config.json -s
```

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a status
```

Output confirmed:


```
{
  "status": "running",
  "configstatus": "configured",
  "version": "1.300072.0b1766"
}
```

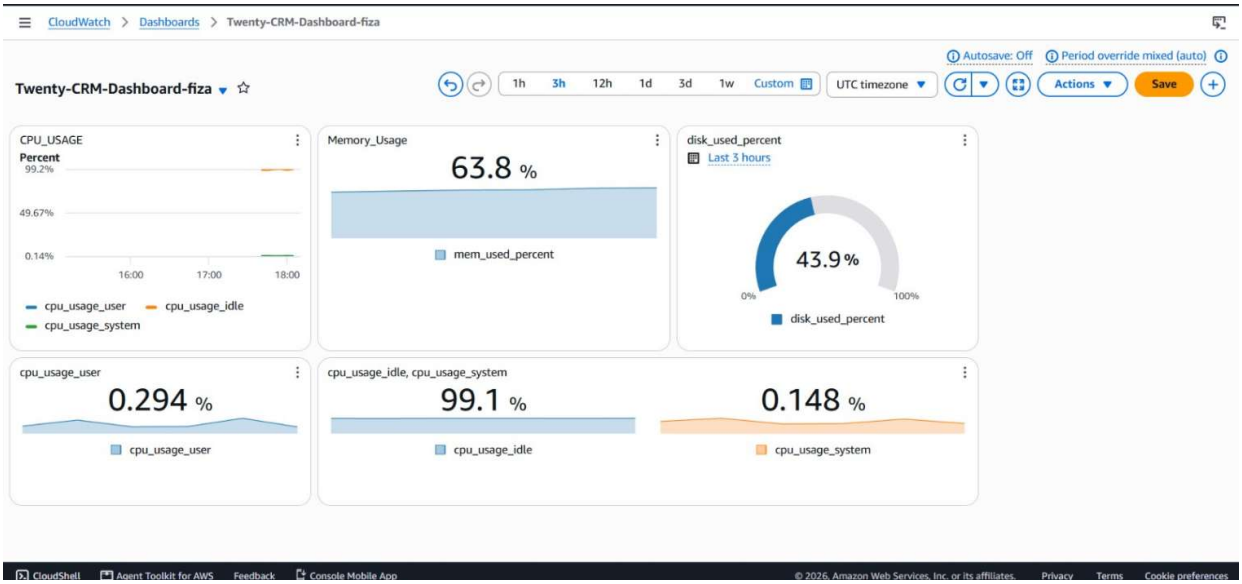

3.7 Verify CloudWatch Agent metrics in CloudWatch

CloudWatch → Metrics → All metrics → custom namespace **TwentyCRM/EC2** appeared after ~2–3 minutes.

Drilled into each metric group:

- cpu, host → cpu_usage_idle, cpu_usage_user, cpu_usage_system
- host → mem_used_percent
- device, fstype, host, path → disk_used_percent

Selected and graphed each to confirm real data points were arriving every 60 seconds.



3.8 Create CloudWatch Alarms

Alarm — High CPU, Disk Usage, Memory Usage

- Metric: EC2 → Per-Instance Metrics → CPUUtilization (and later corroborated with the agent's cpu_usage_active)
- Condition: Static, Greater than **70%**,50%,60% 1 datapoint within 1 evaluation period
- Action: notify via SNS topic (email subscription confirmed)